6.1.2019

CSE 3017 DIGITAL DESIGN
FINAL EXAM SOLUTION MANUAL

Name:

Q1) (15 points) When the below module is simulated with the associated testbench file, what will be the five output (data) values for five consecutive clock cycles?

Note: Enter your results in hexadecimal form to the below table.

```
module Q1(clk, data);

input clk;
output reg [4:0] data;

reg [4:0] data_next;

initial
begin
data = 5'h1f;
end

always @(posedge clk)
begin
  data_next[4] = data[4]^data[1];
  data_next[3] = data[3]^data[0];
  data_next[2] = data[2]^data_next[4];
  data_next[1] = data[1]^data_next[3];
  data_next[0] = data[0]^data_next[2];
  data = data_next;
end

endmodule
```

```
module Q1_tb;

 reg clk;
 wire [4:0] data;

 Q1 UUT (clk, data);

 initial begin
  clk = 0;
  forever
   #20 clk = ~clk;
  end
endmodule
```

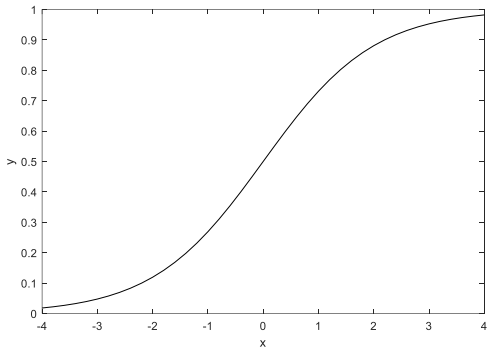
clk	data (hexadecimal form)
20	
40	
60	
80	
100	

SOLUTION:

clk	data (hexadecimal form)
20	06
40	12
60	02
80	17
100	0C

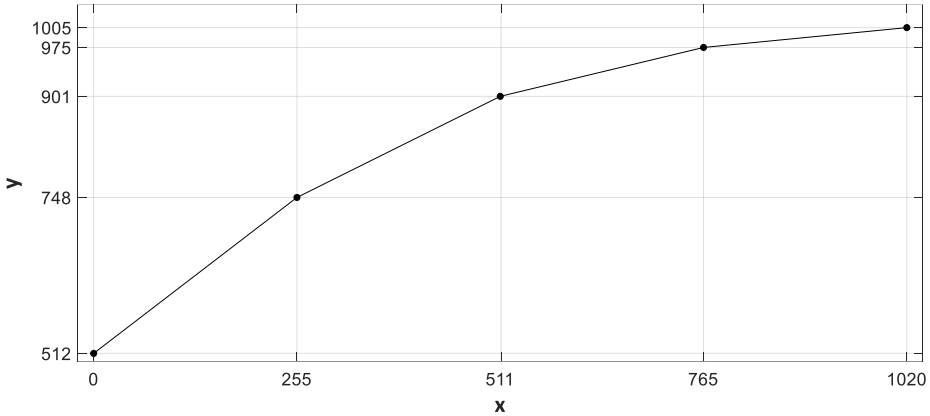
Q2) (40 points) Sigmoid function, given below, is extensively used as a transfer function in neural network implementations.

$$y = \frac{1}{1 + e^{-x}}$$



We would like to implement a transfer function module based on this definition. Since we cannot represent floating-point numbers directly on the FPGA, we will have a fixed-point approximation (with input and output values x, y represented by 12 bits each) as follows.

$$y = \frac{1024}{1 + e^{-x}}$$



a) (15points) Use staircase approximation with <u>case statement</u> based on the below table.
Note: Not using the case statement in this question leads to at most 5 points from this part.

x	y
000 - 255	512
256 - 511	748
512 - 767	901
768 - 1023	975
1024 - …	1005

x (decimal)	x (binary)
000	0000 0000 0000
255	0000 1111 1111
256	0001 0000 0000
511	0001 1111 1111
512	0010 0000 0000
767	0010 1111 1111
768	0011 0000 0000
1023	0011 1111 1111
1024	0100 0000 0000
…	1111 1111 1111

b) (25 points) Use linear approximation for the first four range of values. Again, use the <u>case statement</u> based on the given table.
Note: Not using the case statement in this question leads to at most 15 points from this part.

Hint: For the first range of values (0<x<255), we will have $\frac{y-748}{748-512} = \frac{x-255}{255-0}$. To implement this function, we should not have a negative result in middle steps. Besides, the multiplication operation should be handled with care. Hence, we should have

$$y = 748 - ((748 - 512) * (255 - x))/255$$

SOLUTION:

```
module Q2_a(clk,x,y);

input clk;
input [11:0] x;
output reg [11:0] y;

always @ (posedge clk)
begin

case(x[11:8])
4'b0000 : y=12'd512;
4'b0001 : y=12'd748;
4'b0010 : y=12'd901;
4'b0011 : y=12'd975;
default : y=12'd1005;
endcase

end

endmodule
```

```
`timescale 1ns / 1ps

module Q2_a_tb;

// Input
reg clk;
reg [11:0] x;

// Output
wire [11:0] y;

// Instantiate the Unit Under Test (UUT)
Q2_a UUT(clk,x,y);

//Providing input to the UUT

initial begin
clk = 0;
forever
#5 clk = ~clk ;
end

initial begin
   x=12'd0;
#10 x=12'd254;
#10 x=12'd500;
#10 x=12'd700;
end

endmodule
```

```
module Q2_b(clk,x,y);

input clk;
input [11:0] x;
output reg [11:0] y;

//reg [23:0] dummy;

always @ (posedge clk)
begin

case(x[11:8])
//4'b0000 : begin dummy=236*(255-x); y=748-
dummy/255; end
4'b0000 : y=748-(236*(255-x))/255;
4'b0001 : y=901-(153*(511-x))/256;
4'b0010 : y=975-(74*(765-x))/254;
4'b0011 : y=1005-(30*(1020-x))/255;
```

```
`timescale 1ns / 1ps

module Q2_b_tb;

// Input
reg clk;
reg [11:0] x;

// Output
wire [11:0] y;

// Instantiate the Unit Under Test (UUT)
Q2_b UUT(clk,x,y);

//Providing input to the UUT

initial begin
clk = 0;
```

```
default : y=12'd1005;
endcase

end

endmodule
```

```
forever
#5 clk = ~clk ;
end

initial begin
   x=12'd0;
#10 x=12'd100;
#10 x=12'd350;
#10 x=12'd600;
end

endmodule
```

Q3) (15 points) We would like to construct a Verilog module to multiply two numbers represented in signed bit form. Let's assume that we have the following description part

```
module signed_multiplier(a,b,c);
input [4:0] a,b;
output [8:0] c;
```

As can be seen in this form, input vectors a and b have length five, with the fifth bits (a[4], b[4]) keeping sign bit. If the number is negative, then it is also kept in two's complement form.

The easiest method to calculate the signed bit multiplication is extending the sign bit of each input to output range (here eight bits); perform the multiplication; take the least-significant 8 bits; and add the sign bit to the result (c[8]).

Form a Verilog description based on the above procedure. You can only use dataflow modeling in the solution. In case behavioral modeling is used, the solution will not be graded.

SOLUTION:

```
module signed_multiplier(a,b,c);

input [4:0] a,b;
output [8:0] c;

//sign bit output
assign c[8]=a[4]^b[4];

wire [7:0] aw,bw;
wire [15:0] cw;

//sign extend
assign aw={{4{a[3]}},a[3:0]};
assign bw={{4{b[3]}},b[3:0]};

assign cw=aw*bw;

assign c[7:0]=cw[7:0];

endmodule
```

```
module signed_multiplier_tb;

// Input
reg [4:0] a,b;

// Output
wire [8:0] c;

// Instantiate the Unit Under Test (UUT)
signed_multiplier UUT(a,b,c);

//Providing input to the UUT
initial begin
// Initialize Inputs
//a = 5'b00010;//2
a = 5'b11110;//-2
//b = 5'b00011;//3
b = 5'b11101;//-3
end

endmodule
```

Q4) (50 points) There is a need for quality control for production. One such example is a liquid oil production line such that dark oil is filled to transparent bottles. Due to the nature of the operation, bubbles can form during filling. There is a laser sensor which detects each passing bottle. When a bottle passes in front of the sensor, the quality control operation starts. To do so, we would like to design a control unit with below specifications. Here, assume that the main clock frequency of the used FPGA board in operation is 100 MHz.

a- **(20 points)** There is an illuminator and camera sensor on the two sides of the production line. The camera can scan an area of 30x40 pixels. As we feed a clock with 100/16 MHz to the camera, at each clock cycle, we will receive a pixel value in eight bits in between 0 and 255 (0 being black, 255 being white). If the received pixel value is less than 200, then we will assume the corresponding pixel area is full. Hence we should provide logic level 1. Otherwise, we should provide logic level 0.

   i) **(5 points)** Form a frequency divider module which will get 100 MHz clock and generate 100/16 MHz clock.

   ii) **(15 points)** Form a camera control module triggered by the laser sensor output (which is a pulse with short duration) and clock signal. The module will get the eight-bit pixel value from the camera and give one-bit output (corresponding to the emptiness or fullness of the corresponding pixel area) for consecutive 30x40 pixels.

b- **(15 points)** The acceptable rate of the bottle to be full is 80%. If the dark oil in the bottle is below this level, then it should be moved out of the production line. Form a bottle control unit to decide on the level of the bottle controlled.

c- **(15 points)** Moving the empty bottle out of the production line can be done by a moving flap which can be operated by a simple solenoid. A solenoid is an electromechanical device which has a mechanical part moving forward when logic level 1 (Vcc volts) is fed to it. If the input of the solenoid is kept at logic level 0 (0 volts), then it turns back to its original position. The time needed for the flap to push the unfilled bottle out of the production line is 2.1 sec. Form a solenoid control unit such that when a control command comes (as logic level 1), it gives logic level 1 for approximately 2.1 sec (2.097152 sec to be exact). Afterwards, it will give logic level 0 till the next control command.

Hint:

$100*10^6/16=12.5*10^6$
$16^4=65536$
$12.5*10^6/65536=190.73486328125$
$190.73486328125*2.097152=400$

SOLUTION:

Part a)
i)

```
module freq_div(clk_o,clk);

input clk_o;//original clock signal at 100 MHz
output reg clk;//generated clock signal at 100/16 MHz

reg [3:0] F;

initial
F=4'b0000;

always @ (negedge clk_o)
begin
F <= F + 1'b1;
clk<=F[3];
end
endmodule
```

```
`timescale 1ns / 1ps

module freq_div_tb;

// Input
reg clk_o;

// Output
wire clk;

// Instantiate the Unit Under Test (UUT)
freq_div UUT(clk_o,clk);

//Providing input to the UUT

initial begin
clk_o = 0;
forever
#5 clk_o = ~clk_o ;
end

//initial begin
//end

endmodule
```

Part a)
ii)

```
module
camera_module(clk,laser_trig,pix_in,val);

input clk;
input laser_trig;
input [7:0] pix_in;

output reg val;

reg toggle;
reg [10:0] count;

initial
begin
toggle=1'b0;
count=10'd0;
end
```

```
`timescale 1ns / 1ps

module camera_module_tb;

// Input
reg clk;
reg laser_trig;
reg [7:0] pix_in;

// Output
wire val;

// Instantiate the Unit Under Test (UUT)
camera_module UUT(clk,laser_trig,pix_in,val);

initial begin
clk = 0;
forever
```

```
always @ (negedge clk)
begin
if (laser_trig==1'b1)
   toggle<=~toggle;

if (toggle==1'b1)
 begin
   count<=count+1;
   val<=~(pix_in/200);

   if (count==1200)
      begin
      toggle<=~toggle;
      count<=10'd0;
      end
   end
end
endmodule
```

```
#5 clk = ~clk;
end

initial begin
pix_in=8'd220;
#6 laser_trig = 1'b1;
#10 laser_trig = 1'b0;

#800 pix_in=8'd120;

end

endmodule
```

Part b)

```
module
bottle_control(clk,laser_trig,val,full,toggle,count);

input clk;
input laser_trig;
input val;

output reg full;

output reg toggle;
output reg [10:0] count;
reg [10:0] fcount;

initial
begin
toggle=1'b0;
count=10'd0;
fcount=10'd0;
full=1'b0;
end

always @ (negedge clk)
begin
if (laser_trig==1'b1)
   toggle<=~toggle;

if (toggle==1'b1)
 begin
   fcount<=fcount+val;
   count<=count+1;
```

```
`timescale 1ns / 1ps

module bottle_control_tb;

// Input
reg clk;
reg laser_trig;
reg val;

// Output
wire full;
wire toggle;
wire [10:0] count;

// Instantiate the Unit Under Test (UUT)
bottle_control
UUT(clk,laser_trig,val,full,toggle,count);

initial begin
clk = 0;
forever
#5 clk = ~clk;
end

initial begin
val= 1'b1;
#6 laser_trig = 1'b1;
#10 laser_trig = 1'b0;

#800 val= 1'b0;
```

```
   if (count==1200)
      begin
      toggle<=~toggle;
      count<=10'd0;
      full<=fcount/960;
      end
   end
end
endmodule
```

```
end

endmodule
```

Part c)

```
module solenoid_module(clk_in,control,move);

input clk_in; //original 100 MHz clock
input control;

output reg move;

reg [9:0] count;

initial
begin
move=1'b0;
count=9'd0;
end

wire clk1,clk2,clk3,clk4;
wire clk;

freq_div C1(clk_in,clk1);//divide by 16
freq_div C2(clk1,clk2);//divide by 16
freq_div C3(clk2,clk3);//divide by 16
freq_div C4(clk3,clk4);//divide by 16
freq_div C5(clk4,clk);//divide by 16

always @ (negedge clk)
//always @ (negedge clk_in)
begin
if ((control==1'b1)&&(count==9'd0))
   move=~move;

if (move==1'b1)
 begin
   count<=count+1;

   if (count==401)
      begin
      move<=~move;
```

```
`timescale 1ns / 1ps

module solenoid_module_tb;

// Input
reg clk_in;
reg control;

// Output
wire move;

// Instantiate the Unit Under Test (UUT)
solenoid_module UUT(clk_in,control,move);

initial begin
clk_in = 0;
forever
#5 clk_in = ~clk_in;
end

initial begin
#10 control = 1'b1;
//#100 control = 1'b0;
end

endmodule
```

```
      count<=9'd0;
      end
   end
end
endmodule
```